MODULE 4 · TEXTBOOK CHAPTER 18 · 8 HOURS

Interactive Input Methods & GUIs

How a user talks to a graphics program: logical input devices, the three input modes, picture construction techniques, virtual reality, OpenGL input and menu callbacks, and the principles of good user interface design.

Logical Input Devices | Request · Sample · Event | Rubber Band & Dragging | GLUT Input & Menus

Module Roadmap

From the idea of input data to the design of a full interface

Part A · Concepts of Input

- Graphical input data
- Logical classification of input devices
- Input functions: request, sample, event
- Interactive picture construction techniques
- Virtual reality environments

Part B · OpenGL and GUI Design

- OpenGL interactive input device functions
- Mouse, motion, keyboard and special key callbacks
- OpenGL menu functions and submenus
- Designing a graphical user interface
- User model, feedback, consistency, icons

Graphical Input Data

The different kinds of values a graphics program reads from the user

An interactive graphics program must accept many kinds of information: a screen position for a point, a numeric value for a size, a text label, or a choice from a menu. Rather than tie the program to a specific piece of hardware, graphics packages describe input by **what the data means**, not by the device that produced it.

Positions

Screen coordinates for placing points, lines and objects.

Scalar values

Numbers for scaling, rotation angles or intensity.

Text strings

Labels, file names and annotations typed by the user.

Selections

Menu items, buttons or objects chosen from the picture.

DEVICE INDEPENDENCE

By classifying input logically, the same program can work whether the user has a mouse, a tablet, a trackball or a touch screen.

Logical Classification of Input Devices

Six logical classes describe input by the kind of value returned

Graphics standards define **logical input devices**. A logical device is not a physical gadget; it is a category defined by the type of data it produces. One physical device, such as a mouse, can play the role of several logical devices.

Logical class	Value returned	Typical physical devices
Locator	A single coordinate position (x, y)	Mouse, tablet, joystick, trackball
Stroke	A sequence of coordinate positions	Mouse or tablet dragged along a path
String	A text string of characters	Keyboard
Valuator	A single scalar (numeric) value	Dial, slider, scroll bar
Choice	A selection from a set of options	Function keys, buttons, menus
Pick	An object or component of the picture	Mouse click on a displayed object

The Six Logical Devices

Each class answers a different kind of question

Locator

Where? Returns one position in the picture, such as a mouse click point.

Stroke

Along what path? Returns a list of positions traced by dragging.

String

What text? Returns characters typed on the keyboard.

Valuator

How much? Returns a single number from a dial or slider.

Choice

Which option? Returns a selection from a menu or button set.

Pick

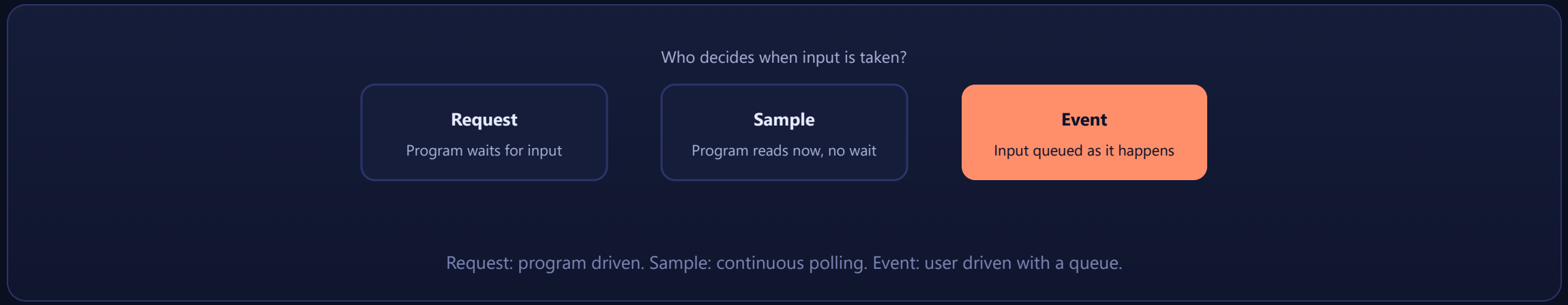
Which object? Returns the identity of a displayed picture element.

KEY IDEA

Locator and Pick both use a screen position, but Locator returns the raw coordinate while Pick returns the object found at that coordinate.

Input Functions for Graphical Data

Three modes decide who controls the timing of input: the program or the user



1. **Request mode**: the program halts and waits until the user provides input from the chosen device, like a blocking read.
2. **Sample mode**: the program reads the current device value immediately without waiting, useful for tracking a continuously changing position.
3. **Event mode**: several devices are active at once; each action is placed in an **event queue** that the program processes in order.

Request vs Event Mode

Two opposite philosophies of control

Request Mode

- Program controls the timing
- Execution pauses until input arrives
- One device is read at a time
- Simple, like scanf reading a value
- Poor for tracking motion or many devices

vs

Event Mode

- User controls the timing
- Program never blocks, it reacts
- Many devices active together
- Actions stored in an event queue
- Basis of modern GUI and OpenGL callbacks

SAMPLE MODE

Sample sits between the two: it neither waits nor queues, it just reports whatever the device value is at the instant of the call.

Interactive Picture Construction

Aids that help the user place and shape objects accurately

When a user draws with a mouse, raw positions are imprecise. Interactive construction techniques give the user assistance so that shapes line up neatly and are easy to build.

Basic positioning

Point at a location and the program reads that coordinate.

Constraints

Force lines to be horizontal, vertical or at fixed angles.

Grids

A background grid the cursor snaps to for even spacing.

Gravity field

Cursor is pulled to a nearby line or endpoint for exact joins.

Rubber band

A shape stretches with the cursor until the user fixes it.

Dragging

Move an existing object smoothly with the pointer held down.

Constraints, Grids and Gravity Field

Three aids that turn rough input into precise geometry

Constraints

The program adjusts input to obey a rule. A common one aligns a line to the nearest horizontal or vertical direction so it comes out perfectly straight.

Grid

A regular pattern of points. The cursor **snaps** to the nearest grid point, so objects share consistent spacing and alignment.

Gravity field

An invisible attraction region around lines and vertices. When the cursor enters it, the point jumps to the exact line or endpoint.

Left: cursor snaps to grid dots. Right: a soft field around the line captures the cursor.

Rubber Band Methods

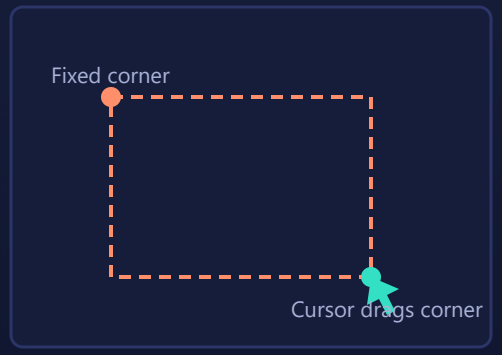
A shape stretches to follow the cursor before it is committed

In a rubber band method the user fixes one point, then moves the cursor. The program continuously redraws a shape anchored at the first point and stretching to the current cursor position. When the user releases, the shape is fixed.

- **Rubber band line**: one endpoint fixed, the other follows the cursor.
- **Rubber band rectangle**: one corner fixed, the opposite corner follows.
- **Rubber band circle**: centre fixed, radius grows with the cursor.

HOW IT WORKS

The old outline is erased and a new one drawn on every mouse move, so the shape appears to stretch like an elastic band.



One corner is anchored; the opposite corner tracks the cursor as the rectangle stretches.

Dragging, Painting and Drawing

Moving objects and laying down colour directly

Dragging

The user selects an object, holds the mouse button, and moves the pointer. The object is repeatedly erased and redrawn at the new cursor position, so it appears to slide across the screen until the button is released.

Painting and drawing

The cursor acts like a brush or pen. As it moves, colour is applied to the pixels it passes over. Brush size, colour and pattern are chosen as attributes, and the stroke follows the cursor path continuously.

Left: object follows the pointer. Right: colour is applied along the cursor path.

Virtual Reality Environments

Immersive input and output that place the user inside the scene

A virtual reality system surrounds the user with a computer generated world. Special input and output devices track the user's head and hands so that the view and the interaction feel natural and three dimensional.

Head mounted display

Two small screens, one per eye, give a stereoscopic 3D view.

Head tracking

Sensors report head position and orientation to update the view.

Data gloves

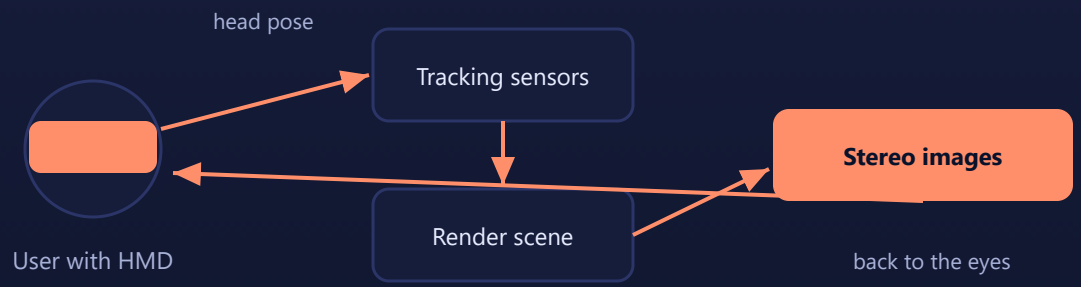
Sense finger bending and hand position for grabbing objects.

3D sound

Spatial audio adds direction and distance cues to the scene.

How VR Tracking Works

The loop that keeps the virtual view matched to the real user



Head pose is measured, the scene is rendered from that viewpoint, and stereo images are sent back to the display many times per second.

LOW LATENCY MATTERS

The whole loop must finish in a few milliseconds, otherwise the view lags behind head motion and the user feels discomfort.

PART B · OPENGL INPUT AND MENUS

OpenGL Input, Menus & GUI Design

Registering mouse, motion, keyboard and special key callbacks with GLUT, building menus and submenus, and applying the principles of good user interface design.

OpenGL Interactive Input Functions

GLUT uses event mode: you register callbacks and it calls them for you

Core OpenGL has no input handling. GLUT fills this gap with callback registration functions. You give GLUT a function to call for each kind of event, then start the event loop with `glutMainLoop()`.

GLUT function	Fires when
`glutMouseFunc`	A mouse button is pressed or released
`glutMotionFunc`	The mouse moves with a button held down
`glutPassiveMotionFunc`	The mouse moves with no button held
`glutKeyboardFunc`	A standard ASCII key is pressed
`glutSpecialFunc`	A special key such as an arrow or function key is pressed

Mouse and Motion Callbacks

glutMouseFunc for clicks, glutMotionFunc for dragging

```
void mouseButton(int button, int state, int x, int y) {
    if (button == GLUT_LEFT_BUTTON && state == GLUT_DOWN) {
        // x, y are the click position in window coordinates
        addPoint(x, y);
        glutPostRedisplay();              // ask for a redraw
    }
}

void mouseDrag(int x, int y) {
    // called repeatedly while a button is held and the mouse moves
    updateRubberBand(x, y);
    glutPostRedisplay();
}

int main(int argc, char** argv) {
    // ... window setup ...
    glutMouseFunc(mouseButton);           // register click handler
    glutMotionFunc(mouseDrag);            // register drag handler
    glutMainLoop();
```

BUTTON AND STATE VALUES

Buttons: `GLUT_LEFT_BUTTON`, `GLUT_MIDDLE_BUTTON`, `GLUT_RIGHT_BUTTON`. State: `GLUT_DOWN` or `GLUT_UP`.

Keyboard and Special Key Callbacks

glutKeyboardFunc for ASCII keys, glutSpecialFunc for arrows and function keys

```
void keyInput(unsigned char key, int x, int y) {
    switch (key) {
        case 'r': setColor(1.0, 0.0, 0.0); break;  // red
        case 27:  exit(0);                  break;  // 27 = Esc key
    }
    glutPostRedisplay();
}

void specialKeys(int key, int x, int y) {
    switch (key) {
        case GLUT_KEY_LEFT:  moveShape(-5,  0); break;
        case GLUT_KEY_RIGHT: moveShape( 5,  0); break;
        case GLUT_KEY_UP:    moveShape( 0, -5); break;
        case GLUT_KEY_DOWN:  moveShape( 0,  5); break;
    }
    glutPostRedisplay();
}
```

TWO KINDS OF KEYS

Printable keys arrive as an `unsigned char` in `glutKeyboardFunc`. Non ASCII keys such as arrows and F1 to F12 arrive as GLUT constants in `glutSpecialFunc`.

OpenGL Menu Functions

GLUT can attach a pop-up menu to a mouse button

1. `glutCreateMenu(fn)` creates a menu and names the callback that receives the chosen entry's id.
2. `glutAddMenuEntry(label, id)` adds one item; the id is passed to the callback when that item is picked.
3. `glutAttachMenu(button)` links the menu to a mouse button, usually the right button.

```
void mainMenu(int item) {
    switch (item) {
        case 1: drawMode = LINES;     break;
        case 2: drawMode = RECT;      break;
        case 3: exit(0);              break;
    }
    glutPostRedisplay();
}

// build the menu inside main()
glutCreateMenu(mainMenu);
glutAddMenuEntry("Draw Lines", 1);
glutAddMenuEntry("Draw Rectangle", 2);
glutAddMenuEntry("Quit", 3);
```

Building Submenus

A submenu is created first, then added into a parent menu

```
void colorMenu(int c) {
    setColor(c);
    glutPostRedisplay();
}

void mainMenu(int item) {
    if (item == 9) exit(0);
}

int main(...) {
    // create the submenu first
    int sub = glutCreateMenu(colorMenu);
    glutAddMenuEntry("Red",   1);
    glutAddMenuEntry("Green", 2);
    glutAddMenuEntry("Blue",  3);

    // then the main menu that uses it
    glutCreateMenu(mainMenu);
    glutAddSubMenu("Colours", sub);
    glutAddMenuEntry("Quit", 9);
    glutAttachMenu(GLUT_RIGHT_BUTTON);
}
```

Designing a Graphical User Interface

Principles that make an interface easy and pleasant to use

User model

Match the interface to how the user pictures the task, using familiar concepts.

Help and prompts

Offer guidance, tips and clear prompts at several levels of detail.

Feedback

Confirm every action with a visible or audible response so the user is never unsure.

Consistency

Use the same commands, colours and layouts throughout the application.

Minimize memorization

Show options as menus and icons so the user recognizes rather than recalls.

Undo and error recovery

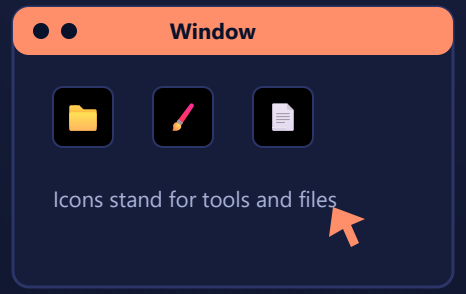
Let users back out of mistakes safely with undo and clear error messages.

Windows, Icons and Feedback

The visible building blocks of a modern GUI

Most graphical interfaces are built from a small set of visual parts, often called the **WIMP** model: windows, icons, menus and a pointer. Good design arranges these clearly and gives constant feedback.

- **Windows** divide the screen into task areas that can be moved and resized.
- **Icons** are small pictures that stand for objects, tools or actions.
- **Menus** and **buttons** present the available commands.
- **Pointer** lets the user select and manipulate items directly.



A window with a title bar, tool icons and a pointer: the familiar WIMP interface.

RECOGNITION OVER RECALL

Module 4 Summary

Concepts of Input

- Input data: positions, values, strings, selections.
- Six logical devices: locator, stroke, string, valuator, choice, pick.
- Three modes: request (wait), sample (poll), event (queue).
- Construction aids: constraints, grids, gravity, rubber band, dragging, painting.
- VR: head mounted display, tracking, data gloves.

OpenGL and GUI Design

- Input callbacks: glutMouseFunc, glutMotionFunc, glutKeyboardFunc, glutSpecialFunc.
- Menus: glutCreateMenu, glutAddMenuEntry, glutAttachMenu, glutAddSubMenu.
- GUI principles: user model, help, feedback, consistency.
- Minimize memorization with windows, icons and menus.

NEXT UP

Module 5, Computer Animation, Chapter 11.

Sources & References

Textbook chapter and official documentation for this module

- Hearn, Baker & Carithers, **Computer Graphics with OpenGL**, 4th ed., Pearson, 2014, **Chapter 18** (Interactive Input Methods and Graphical User Interfaces).
- GLUT input and menu callbacks: GLUT documentation, OpenGL reference pages.
- Reference: Edward Angel, **Interactive Computer Graphics**, 5th ed., Pearson, 2008.

LECTURER

Mr. Bharath · B24CS53 Computer Graphics.